Business Requirements Document (BRD)

Project: Pulse - Music Streaming Platform

1. Document Overview

This document outlines the business and technical requirements for the **Pulse Music Streaming Platform**, a backend-driven application built with **Java Spring Boot**. It serves as the primary reference for stakeholders and developers to ensure alignment on features, goals, and constraints.

2. Project Scope

The goal of the Pulse project is to create a scalable, secure, and high-performance music streaming service.

- **In-Scope:** User authentication, song metadata management, audio streaming (Partial Content), playlist creation, and admin file uploads.
- **Out-of-Scope:** Real-time social chat between users, video streaming, and physical merchandise sales.

3. Stakeholders

- **Project Manager:** Oversees timeline and delivery.
- **Backend Developer:** Implements Spring Boot APIs and database logic.
- **DevOps Engineer:** Manages cloud storage (AWS S3) and deployment (Docker).
- **End Users:** Individuals consuming music content.

4. Functional Requirements (FR)

ID	Requirement Name	Description	Priority
FR-01	User Authentication	Users must be able to sign up, log in, and receive a JWT for secure API access.	High
FR-02	Music Streaming	The system must support byte-range requests (HTTP 206) to allow users to "scrub" through songs.	High
FR-03	Metadata Management	The system must store song titles, artist names, album art URLs, and genres in a relational database.	High

FR-04	Search Engine	Users must be able to search for music by keywords (Title, Artist, Album).	Medium
FR-05	Playlist Lifecycle	Users can create, update, and delete personalized playlists.	Medium
FR-06	Admin Upload	Authorized admins must be able to upload `.mp3` files to the storage layer via a protected endpoint.	High
FR-07	Favorite Tracks	Users can "like" a song to add it to a default "Liked Songs" library.	Low

5. Non-Functional Requirements (NFR)

- **Performance:** The API response time for fetching song metadata should be under 200ms.
- **Scalability:** The architecture must support horizontal scaling using Spring Boot's stateless nature.
- **Security:** All passwords must be hashed using `BCrypt`. JWT tokens must expire after 24 hours.
- **Reliability:** The system should handle concurrent streaming requests without audio "stuttering" or server crashes.
- **Data Integrity:** Relationships between Artists and Songs must be enforced via foreign key constraints in PostgreSQL.

6. Technical Architecture & Flow

A. Backend Architecture

- **Framework:** Spring Boot 3.x (Web, Security, Data JPA).
- **Database:** PostgreSQL for relational data (Users, Playlists, Metadata).
- **Storage:** AWS S3 or Local File System for binary audio data.
- **Auth:** Spring Security with JWT (JSON Web Tokens).

B. The Streaming Logic Flow

1. **Client** sends a GET request to `/api/songs/stream/{id}`.
2. **Spring Boot Controller** validates the JWT.
3. **Service Layer** fetches the file location from the DB.
4. **Streaming Component** reads the file and returns a `ResourceRegion` with a `206 Partial Content` status.

5. **Client Player** buffers the chunks and begins playback.

7. Success Criteria

1. Successful deployment of the Spring Boot application to a cloud provider.
2. Successful streaming of a 3-minute MP3 file with seeking functionality.
3. Zero unauthorized access to user-private playlists.

8. Glossary

- **JWT:** JSON Web Token, used for stateless authentication.
- **HTTP 206:** Partial Content status code used for streaming large files.
- **JPA:** Java Persistence API, used to map Java objects to database tables.